

CacheQuant: Faster GPT-2 on CPU

Presenter: Vicky Nguyen

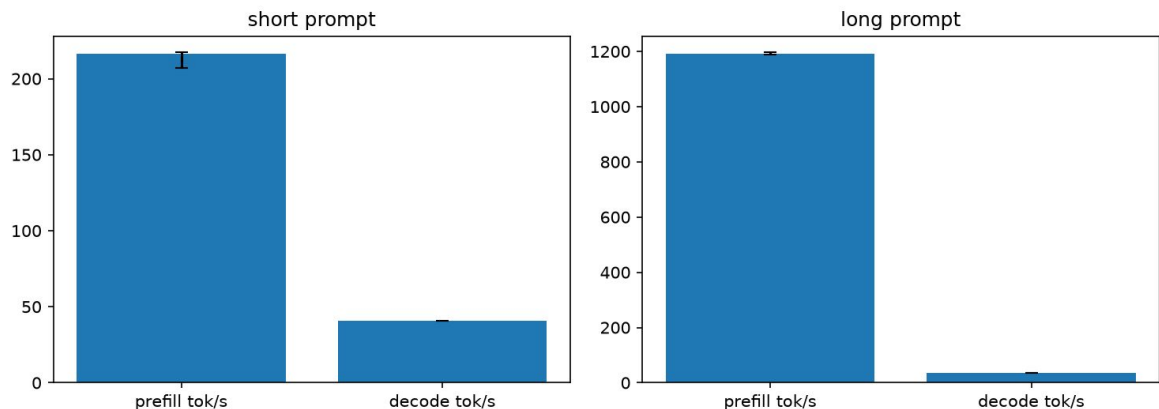
Why Optimize GPT-2 Small for CPU?

- No GPU. Just a normal CPU.
- Picked GPT-2 small – smallest well-known AI text model
- Question: how fast can this run using only what I have?
- Real-world lesson: most people don't have expensive GPUs. Optimizing for that matters.

Setup: What I'm Working With

- Regular CPU, no GPU — Intel i7-8700K, 12 threads. Pin 8 threads for every run.
- No native fp16 - it can convert fp16 <-> fp32, but every fp16 multiply happens in fp32 under the hood.
- Pinned: torch 2.13, transformers 5.14, numpy 2.4, numba 0.66, Python 3.14.
- Running GPT-2 small (124M params) fully on CPU
- Pricing: AWS c7i.2xlarge 8vCPU pricing (\$0.357/hr) - just use the number for reference

Baseline throughput (median, min-max range across reps)



Baseline speed (GPT2-small, fp32), median across 5 runs:

- Short prompt (~8 tokens): prefill ≈ **216 tok/s**, decode ≈ 41 tok/s
- Long prompt (~200+ tokens): prefill ≈ **1190 tok/s**, decode ≈ 36 tok/s
- ~24ms to produce each new word
- Generating 50 words takes ~1.23s

Two Optimizations to Make It Faster

Optimization 1: Quantization

Make each matrix multiply cheaper by compressing weights and activations from 32-bit floats down to smaller numbers.

Optimization 2: KV Cache

If new request shares text with a past one, reuse the already-computed part instead of redoing it from scratch.

These solve 2 different bottlenecks. Quantization speeds up generating each new word; the cache speeds up handling repeated context.

Optimization 1: Quantization

- A weight in the model is normally a **32-bit float**: 1 sign bit, 8 exponent bits, 23 mantissa bits -> Huge dynamic range, more precision than the model needs.
- Quantization uses **8-bit integer** + **one shared "scale" number per group** -> ~4x fewer bytes to move, and the multiply becomes an integer multiply.
- Why 8 bits, not 4? On this CPU int8 multiply-accumulate is a native instruction and int4 is not. 4-bit would have to be faked in software, which is slower.
- Only compress the model's big multiply-heavy layers (attention + feed-forward).
- 2 quantization schemes and run them side by side. They differ in exactly one thing: **how the scaling factor is chosen**.

Scheme A: BFP8

BFP block (1 shared exponent + int8 mantissas)

Example:

1. block $\max_abs = 1.9$
2. shared exponent = $\lceil \log_2(1.9) \rceil = 1 \rightarrow \text{scale} = 2^1 = 2$
3. value $\rightarrow$ mantissa = $\text{round}(\text{value} / \text{scale} * 127)$

examples:

- value 1.5 $\rightarrow$ mantissa = $\text{round}(1.5/2*127) = 95$
 - value -1.9 $\rightarrow$ mantissa = $\text{round}(-1.9/2*127) = -121$
4. dequant: mantissa * scale / 127
 - $95 * 2/127 = 1.496$ (original was 1.5, close enough)

$\Rightarrow$ Cost per value: 8 bits (mantissa) + 8 bits shared exponent / 32 values = **8.25 bits (vs 32 for fp32)**

Scheme B: int8

One scale per row instead of per 32-block

1. $\text{scale} = \text{max_abs of the row}$ (exact, not rounded)
2. $\text{mantissa} = \text{round}(\text{value} / \text{scale} * 127)$
3. $\text{dequant} = \text{mantissa} * \text{scale} / 127$

=> Cost per value = $8 + (32\text{-bit float scale}) / 768$ values per row $\approx$ **8.04 bits (vs BFP8's 8.25 bits)**

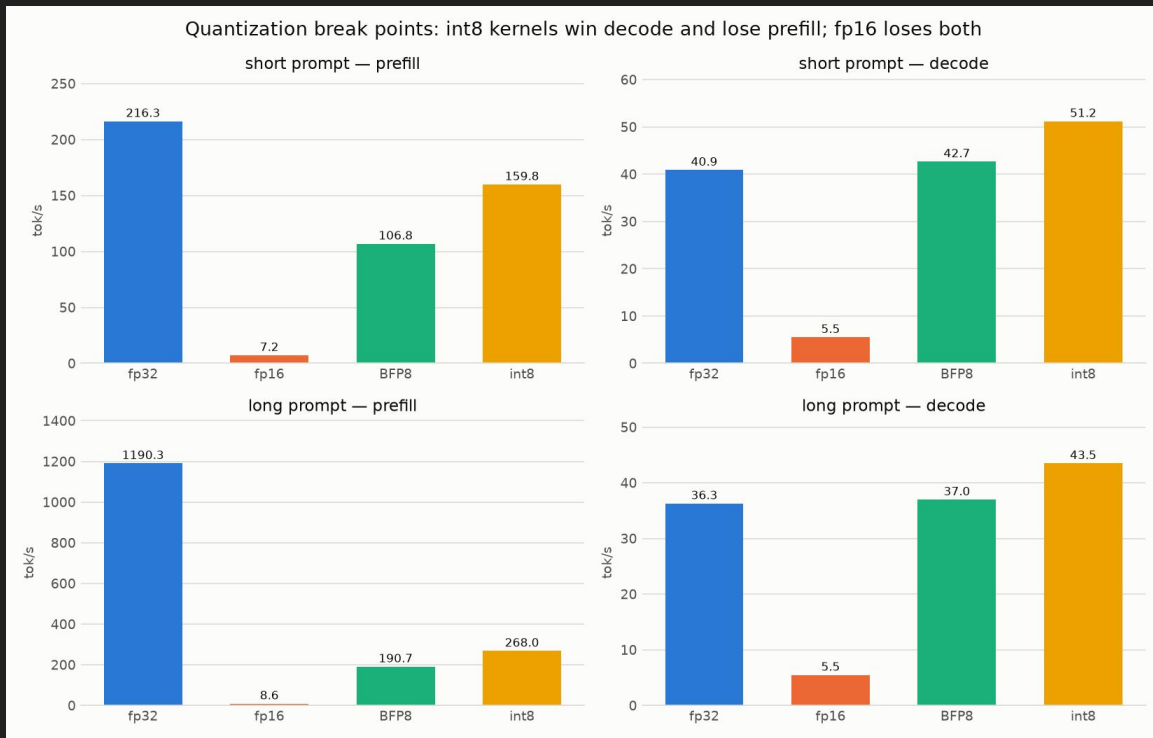
Trade Offs:

- Upside: no wasted range – full use of every bit.
- Downside: one outlier now drags down precision for the entire row, not just a small 32-number group

How I benchmark

- Two prompt profiles, every run:
 - **Short:** ~8-token prompt, 50 generated tokens -> decode-heavy.
 - **Long:** ~270-token prompt, 10 generated tokens -> prefill-heavy.
- **One throwaway warm-up run** first (lets one-time startup costs settle), then 5 timed runs, report the median, not the average, to resist outlier runs.
- **Prefill and decode are timed separately** with their own stopwatches.
- **Quality checked** two ways: a perplexity score (how well the model predicts real text) on held-out passages, plus a side-by-side comparison of actual generated text against the uncompressed model.
- Every result is **tagged** with the exact machine and software versions used, so a number can't get quietly reused after something changes.
- **Correctness check:** the int8 kernel is unit-tested against the fp32 reference with a relative-error bound before any speed number is taken.

Quantization Results: Compression vs. Speed



Short prompt, tokens/sec, % is vs fp32:

Config	Prefill	Decode	Cost/1K	vs fp32
fp32	216 tok/s	41 tok/s	\$0.00244	1.00×
BFP8	107 tok/s	43 tok/s	\$0.00241	0.99×
int8	160 tok/s	51 tok/s	\$0.00198	0.81×
fp16	7 tok/s	5.5 tok/s	\$0.0196	8.0×

- Quantization **loses at prefill** (49% / 74% of fp32) — always.
- Quantization **wins at decode** (104% / 125% of fp32) — and int8 by a solid margin.
- **int8** is the first configuration in the project to **beat fp32 on cost** (\$0.00198 vs \$0.00244)
- **fp16 is a disaster** here. That's not a bug. Included on purpose, explained next slide.

Why those results: the break point

- **Prefill is compute-bound.**
 - Prefill reads the whole prompt in one big batched matmul. Each weight is loaded from RAM once, then multiplied against every prompt token -> lots of math per byte moved.
 - int8 kernel is a readable triple-loop compiled by Numba; it cannot match Intel MKL's hand-tuned vectorized fp32 kernels -> expected lose.
- **Decode is memory-bandwidth-bound.**
 - The time is spent reading the weights from RAM. int8 mantissas are ~4x smaller than fp32 weights → ~4x less to read → faster.
- **The experiment fp16.**
 - This CPU has no native fp16 multiplier – every fp16 op unpacks to fp32, computes, repacks.
- **Takeaway:** shrinking numbers only helps if the hardware can actually compute with the shrunk format directly. int8 works because CPU's instruction set has a built-in multiply-and-add instruction for 8-bit integers.



decode tok/s vs perplexity cost. fp16 sits far bottom-left (slow, no quality gain). int8 sits top, slightly right.

How quantization is implemented

Files:

cachequant/kernel/

blocked_matmul.py ← the ONE shared compiled kernel both schemes call

bfp.py, bfp_numba.py, bfp_linear.py ← scheme A: pick scale, call kernel

int8.py, int8_numba.py, int8_linear.py ← scheme B: pick scale, call kernel

cachequant/kernel/int8.py:

```
max_abs = np.max(np.abs(x), axis=-1)      # one scale per row, exact
```

```
mantissa = np.round(x / max_abs[...], None) * 127.0)
```

cachequant/kernel/bfp.py:

```
max_abs = np.max(np.abs(blocked), axis=-1) # one scale per 32-value block
```

```
exponent = np.ceil(np.log2(max_abs))        # rounded up to power of two
```

```
scale = np.exp2(exponent)
```

```
mantissa = np.round(blocked / scale[...], None) * 127.0)
```

The one shared kernel (blocked_matmul.py), both schemes call this unchanged:

```
@njit(cache=True, parallel=True)
```

```
def blocked_int8_matmul_kernel(a_mantissa, a_scale, b_mantissa, b_scale, bias):
```

```
    for j in prange(n):          # parallel over output features
```

```
        for i in range(m):
```

```
            acc = 0.0
```

```
            for blk in range(num_blocks): # BFP: 32-wide blocks. int8: 1 block = whole row
```

```
                dot = np.int32(0)
```

```
                for k in range(block_size):
```

```
                    dot += np.int32(a_mantissa[i, blk, k]) * np.int32(b_mantissa[j, blk, k])
```

```
                acc += dot * a_scale[i, blk] * b_scale[j, blk]
```

```
            out[i, j] = acc * INV_127_SQUARED + bias[j] # bias fused into the store
```

Optimization 2: KV-cache prefix reuse

- "KV cache" means three different things:
 - **Decode KV cache** (within one generation): store K/V of tokens you've already generated so the next token is $O(1)$. Universal, always on, not what we built.
 - **Batching / continuous batching** (serve many requests at once, pack them into one GPU): also not this. We're batch size 1.
 - **Cross-request prefix reuse (this project)**: when a new request begins with the same tokens as an earlier request, reuse the earlier request's K/V for that shared prefix.
- Only affects prefill (encoding the prompt). Decode is untouched.

Example (same [system prompt] -> computed once, reused):

- Request 1: [system prompt][question A]
- Request 2: [system prompt][question B]

PrefixKVCache

- PrefixKVCache is a trie (prefix tree) keyed on token IDs. Each node = one token position, holding that position's per-layer K and V tensors. LRU eviction (NIY) and capped at **max_cache_tokens (2048)**.
- On a new request: find the longest cached prefix of the prompt, reuse its K/V, run a forward pass on only the uncached suffix, then insert the new K/V back for next time.
- Hard ceiling: the last prompt token is always recomputed (you need a fresh forward pass there to get next-token logits). So max hit rate for an N-token repeated prompt is $(N-1)/N$, never 1.
- When is it actually a win? The cache has a **fixed cost O** (trie lookup + copying K/V + insert). It only pays if the **prefill work you skip is bigger than O**.
- Each node in the trie holds one word's cached data, not one tensor, but a full set: one (K, V) pair per transformer layer.

- root -> [word1] -> [word2] -> [word3] ...

|

12 layers x (K, V) = 24 tensors stored here

Prefix Reuse Results

Three prompt sets (run_kvcache_benchmark.py):

- **high_reuse**: ~24-token prompts, ~12-token shared preamble.
- **long_high_reuse**: ~282-token prompts, ~280-token shared preamble.
- **no_reuse**: independent prompts, no overlap.

Why high_reuse is flat:

p = prompt tokens, h = hit rate, w = time to prefill one token,

O = fixed cache overhead per request

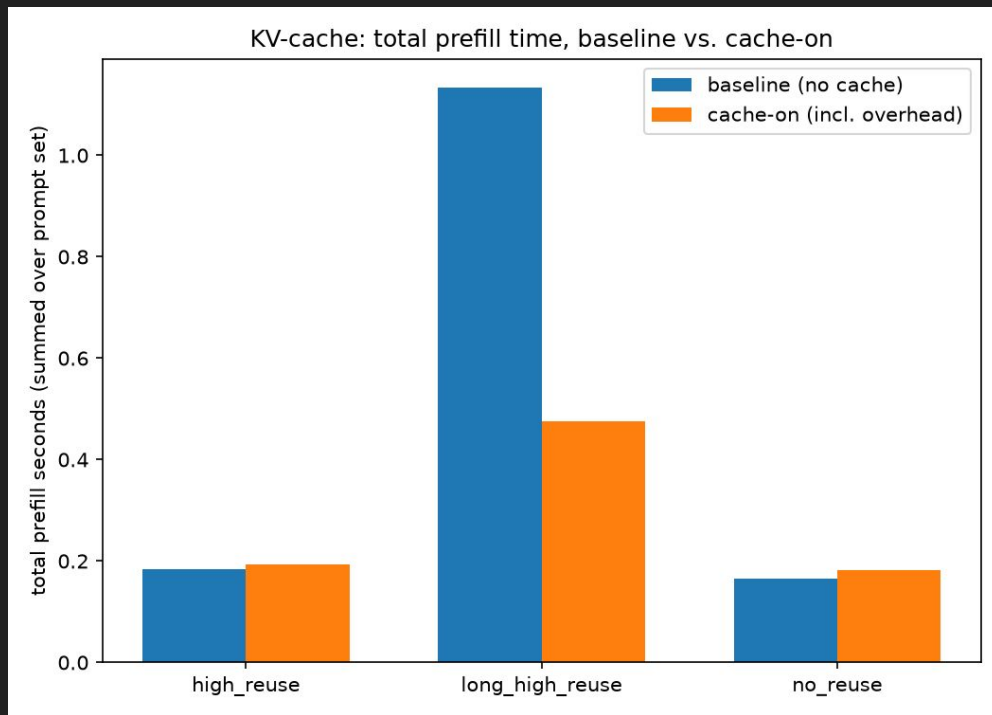
no cache: $p * w$

with cache: $(1 - h) * p * w + O$

cache wins $\Leftrightarrow h * p * w > O$ (roughly)

Plug -> Overhead $O \approx 5$ ms; marginal per-token prefill $w \approx 0.75$ ms; hit rate $h \approx 0.7$; $p \approx 24$. Saving is $h \cdot p \cdot w \approx 13$ ms against $O \approx 5$ ms – barely above break-even, well inside run-to-run noise.

no_reuse shows the downside honestly: if nothing is shared, every request still **pays for a failed lookup and a useless insert**. ~9% slower.



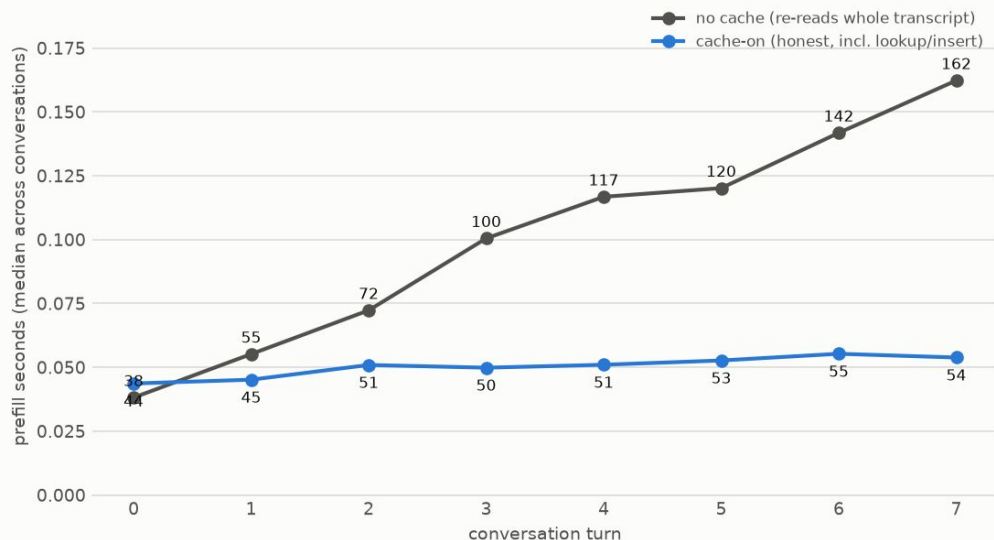
prompt set	shared text	speedup
short shared preamble (~12 words)	~70% reused	~1.0x (no real win – noise)
long shared preamble (~280 words)	~97% reused	4.5x
no shared text	~2% reused	0.90x (a 9% tax)

The Real Use Case - Multi-Turn Chat

- The tests so far send independent, unrelated requests — barely any shared text, so the cache barely helps.
- A conversation is different: each new turn re-sends the entire conversation so far (system instructions + every past message + every past reply) + one new line -> The shared, reusable part keeps **growing every turn** - this is the workload prefix reuse was actually built for.
- Without the cache: every turn re-reads the whole growing conversation from scratch. With the cache: every turn only reads the one new line.

Multi-turn Results

Multi-turn chat: no-cache prefill grows with the transcript, cache-on stays flat



- Median across conversations, per turn
- 3 conversations (2 short system prompt, 1 with a ~280-token system prompt), 8 turns each, 5 reps

turn	no-cache	cache-on	reused	speedup
0 (cold start)	38 ms	44 ms	0%	0.87x
1	55 ms	45 ms	53%	1.22x
3	100 ms	50 ms	75%	2.02x
5	120 ms	53 ms	84%	2.28x
7	162 ms	54 ms	87%	3.01x

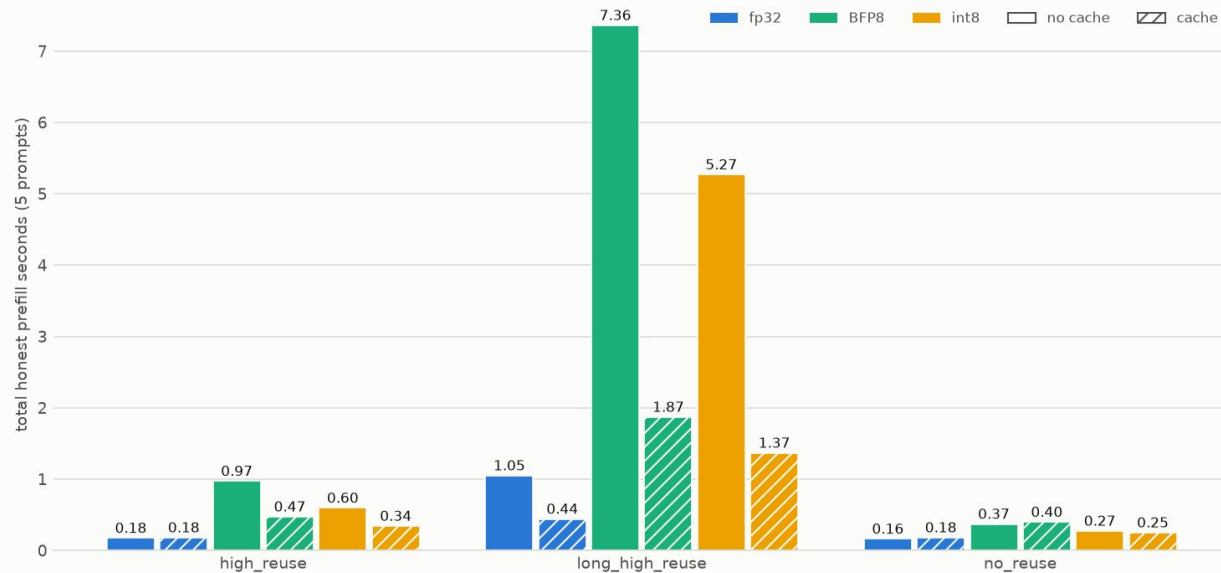
- Turn 0 loses slightly (0.87x), nothing to reuse yet, still pays the storage cost.
- Every turn after, the win grows: without cache, time climbs (38ms -> 162ms) as the conversation gets longer. With cache, time stays roughly flat (~50ms)
- By turn 7, cached chat is **3x faster** than uncached.

PrefixKVCache Limitations

- **Storage is heavy:** each cached word takes ~86KB (12 layers × K/V pairs) -> ~172 MB at the 2048-token cap.
- **Eviction scan is $O(n)$:** walk the whole trie per evicted node -> wouldn't scale to a large production cache.
- Every **cache hit copies the matched data** into a fresh block of memory – a small but real cost per request (~8ms for a long shared prefix).
- On workloads with **no shared text**, the cache still costs ~9% – pure overhead.
- The multi-turn benchmark **never triggers eviction** (longest transcript 489 tokens vs the 2048 cap). Eviction isn't implemented yet.
- **int8 + cache interaction:** int8 compression's exact scale is **sensitive** to tiny rounding differences, so an int8 answer served from cache can occasionally differ. Not a quality problem, just a reproducibility quirk. BFP8 doesn't have this issue.

Combining Both Optimizations

Combined pipeline: the cache is worth more the slower the prefill path is



- Cache behavior is the same regardless of compression.
- The 2 optimizations are independent in performance.

Conclusion

- **Decode = memory-bound -> quantization wins**
 - int8 weights 4x smaller -> **1.25x decode**. No precondition.
- **Prefill = compute-bound -> prefix reuse wins.**
 - Skip the shared prefix entirely -> **3-6x prefill** in multi-turn chat. Needs actual reuse.

Demo

Q&A